

Tortuosity.jl: billion-voxel diffusion simulations in seconds on a GPU, minutes on a CPU

Mohammad Amin Sadeghi¹, Sawyer Hossfeld¹, Harry Kim¹, and Jeff T. Gostick¹

¹ Department of Chemical Engineering, University of Waterloo, Waterloo, Canada  Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 05 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Tortuosity.jl is a Julia package for simulating diffusion directly on segmented two- and three-dimensional images of porous materials. It computes effective diffusivity and the associated tortuosity factor from steady-state solutions and supports transient diffusion and spatially varying diffusivity. The package restricts the numerical problem to the transporting phase and provides both assembled and matrix-free operators. The same interface runs on CPUs and NVIDIA, AMD, or Apple GPUs through Julia's GPU ecosystem. This combination enables image-based transport calculations ranging from small exploratory studies to volumes containing one billion voxels on a single workstation GPU.

Statement of need

Transport through a porous material depends on the connectivity and geometry of its pore space. Quantifying this effect is important in fields including electrochemical energy storage, geoscience, filtration, and membrane science (Fu et al., 2021; Ghanbarian et al., 2013). Three-dimensional imaging allows researchers to estimate transport properties from a resolved microstructure rather than from an empirical porosity correlation. For diffusion through a single transporting phase, the tortuosity factor τ is commonly defined by

$$D_{\text{eff}} = D_0 \frac{\varepsilon}{\tau}$$

where D_{eff} is the effective diffusivity, D_0 is the intrinsic diffusivity of the transporting phase, and ε is its volume fraction. This transport-based quantity is distinct from geometric definitions based on path length (Tjaden et al., 2016). Computing it requires solving a diffusion equation over a voxelized domain and evaluating the resulting flux. Modern images can contain hundreds of millions of voxels, making memory use and solver convergence central practical constraints, especially when a study compares many samples or operating conditions. Existing packages support parts of this workflow, but none combines a pore-only formulation, matrix-free GPU execution, and integration with Julia's numerical ecosystem. Tortuosity.jl targets researchers and engineers in academia and industry who need these calculations to run efficiently on available CPU or GPU hardware while retaining control over the numerical workflow.

State of the field

Several established packages calculate tortuosity from voxel images. TauFactor introduced an accessible MATLAB implementation based on successive over-relaxation (Cooper et al., 2016),

36 and its Python successor, `taufactor`, uses PyTorch to support GPU execution (Kench et al.,
37 2023). Both update the full image grid, including solid voxels. PuMA provides a broader suite
38 of porous-material properties and solves its finite-volume diffusion problem on the full grid
39 (Ferguson et al., 2018, 2021). PoreSpy instead constructs and solves a pore-only network using
40 algebraic multigrid, but its finite-difference tortuosity solver is CPU-only (Gostick et al., 2019).

41 `Tortuosity.jl` combines a pore-only formulation, matrix-free operation, preconditioned Krylov
42 solvers, and portable GPU execution. Restricting the unknowns to the transporting phase is
43 particularly beneficial at low porosity, while the matrix-free path reduces the memory required
44 for large, open structures. Adding this combination to `TauFactor` or `taufactor` would require
45 replacing their central full-grid relaxation method; adding it to `PoreSpy` would require a
46 GPU-compatible operator and solver stack outside its network-based model. Implementing
47 the approach as a Julia package also permits direct integration with Julia's linear-solver and
48 differential-equation ecosystems (Kimmerer et al., 2024; Rackauckas & Nie, 2017).

49 Software design

50 The architecture separates porous-media problem construction from numerical solution and
51 post-processing (Figure 1). Users provide a segmented image and simulation choices; the
52 problem constructors validate and index the geometry, apply boundary conditions, and select
53 the compute backend. The steady path creates a linear problem, while the transient path
54 creates a diffusion right-hand side for time integration. Both paths feed a common analysis
55 layer for transport properties and concentration-field measurements. This separation keeps
56 image and transport logic within `Tortuosity.jl` while allowing established Julia packages to
57 supply solvers and hardware backends.

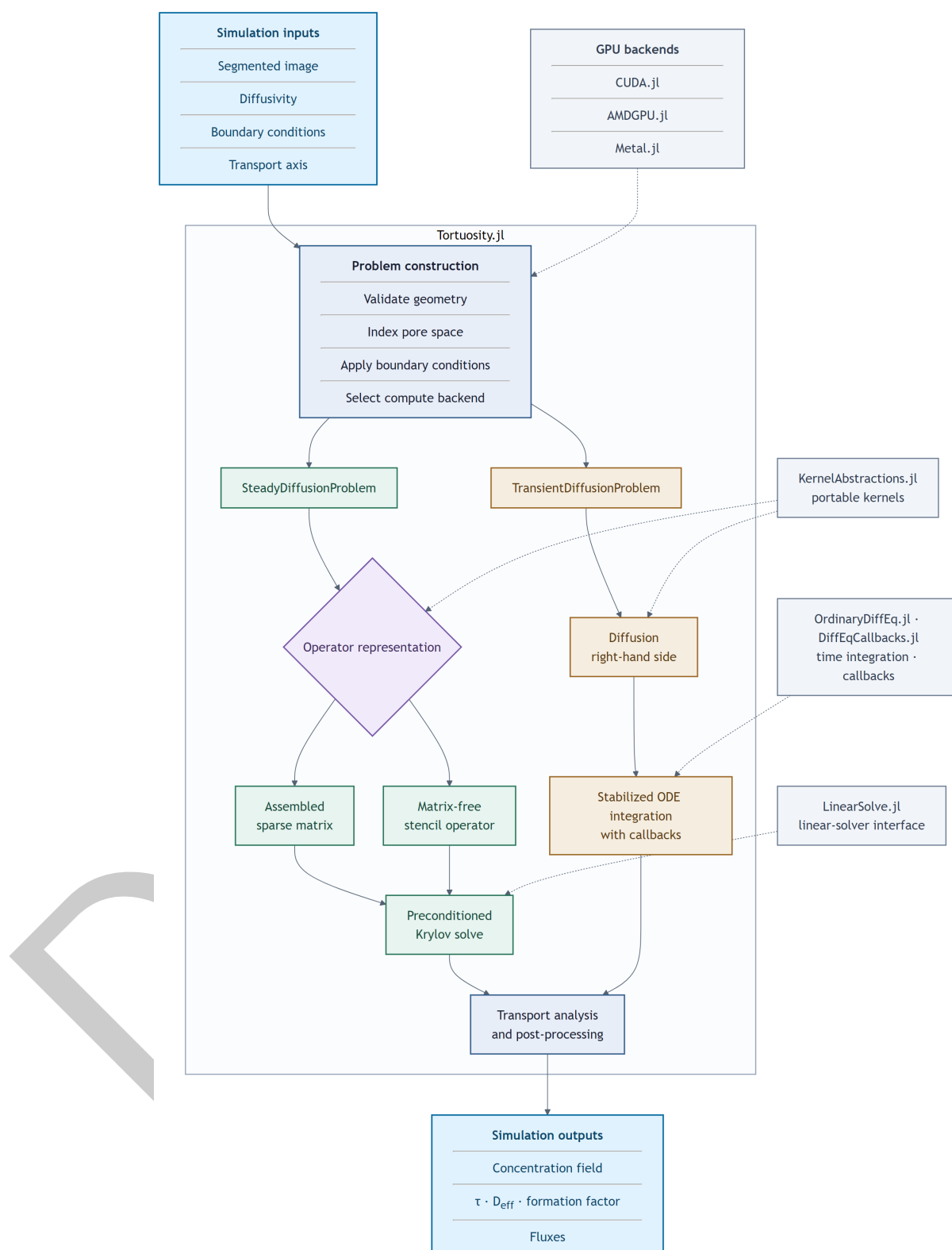


Figure 1: High-level architecture of Tortuosity.jl. Solid arrows show the flow from user inputs to simulation outputs; dotted arrows show interfaces to principal numerical dependencies and optional GPU backends. Colors distinguish inputs and outputs, shared stages, steady and transient paths, the operator choice, and external dependencies.

Two interchangeable operator representations expose the main design trade-off. The assembled representation stores a sparse matrix and can use vendor sparse-linear-algebra libraries. The matrix-free representation stores only a four-byte index map over the image and reconstructs each row during a matrix-vector product, exchanging repeated arithmetic for lower storage and data movement. A geometric coarse-space preconditioner supplies long-range corrections that conjugate gradient would otherwise recover slowly; over the measured 200^3 to 600^3 range, it reduced iteration counts by factors of 12.5 to 29.6.

The numerical engines are integrated through their native interfaces rather than reimplemented, with one deliberate exception: because the host steady iteration is limited by memory bandwidth rather than arithmetic, the package registers its own `LinearSolve.jl` conjugate-gradient algorithm, `HostCG`, which fuses the vector kernels of the standard preconditioned iteration by combining the solution and residual updates and folding the preconditioner prolongation into the residual inner product. It is selected automatically for double-precision host systems above ten thousand unknowns, while smaller host problems and all GPU problems use the generic Krylov solver. Steady problems otherwise use `LinearSolve.jl`, and transient problems use `OrdinaryDiffEq.jl` with composable callbacks (Kimmerer et al., 2024; Rackauckas & Nie, 2017). Data-parallel kernels are implemented with `KernelAbstractions.jl` (Churavy & contributors, 2024), while package extensions register CUDA, AMDGPU, or Metal without making them core dependencies. The CUDA path additionally uses `cuSPARSE` for assembled matrix-vector products (Besard et al., 2019). Other extensions keep image filtering, curve fitting, and HDF5 export optional. GPU calculations use single precision for performance and portability, with iterative refinement against a double-precision residual when additional accuracy is needed.

Validation and performance

Validation combines analytical tests with comparisons against independent software. The test suite covers geometries with closed-form solutions, including straight ducts, layered media, and dead-end pores; steady-state results agree with these solutions to a relative error of 10^{-8} or better on the CPU. Cross-code comparisons against `taufactor`, `PuMA`, and `PoreSpy` show agreement at approximately the benchmark's 0.1% accuracy scale, with tighter agreement when the comparator's available settings resolve that target. The assembled and matrix-free operators agree within the recorded precision in double-precision calculations and typically differ by about 6×10^{-5} in single-precision GPU calculations.

Performance was evaluated on synthetic images ranging from 200^3 to 1000^3 voxels and covering five porosities and three characteristic feature sizes. Timings included the complete workflow from image input through problem construction and solution, and tools were compared at a common target of at most 0.1% relative error in τ . On the GPU, `Tortuosity.jl` was faster in all 70 paired cases spanning five porosities, three feature sizes, and five image sizes, by a pooled geometric mean of $8.4\times$; the narrowest margin was $1.3\times$, at $N = 200$, where end-to-end times for both tools were about one second or less. For the reference feature size shown in Figure 2, the narrowest cell was $N = 200$ at $\varepsilon = 0.95$, where the reference tortuosity was 1.04 and the end-to-end times were 0.46 s for `taufactor` and 0.27 s for `Tortuosity.jl`. At the largest size, $N = 1000$, `Tortuosity.jl` was faster at every porosity where both tools reached the accuracy target; at $\varepsilon = 0.4$, for example, it required 21 s, compared with approximately 12 minutes for `taufactor`. Across the twelve porosity–feature-size combinations for which both tools reached the target at all five image sizes, `Tortuosity.jl` was faster by a geometric mean of $9.1\times$ at 1000^3 . On the CPU at 200^3 , `HostCG` was faster than both CPU-only comparators in every shared case, with geometric-mean speedups of $105\times$ over `PuMA` and $34\times$ over `PoreSpy`. Against `taufactor` on the CPU it was faster by a geometric mean of $10.3\times$ over the 56 paired cases both tools completed; `taufactor` was faster in five, all at $N = 200$ and $\varepsilon \geq 0.8$, and from 400^3 onward `Tortuosity.jl` was faster in every paired CPU case. The matrix-free representation also completed every percolating 1000^3 test image on a GPU with 48 GiB of

109 memory.

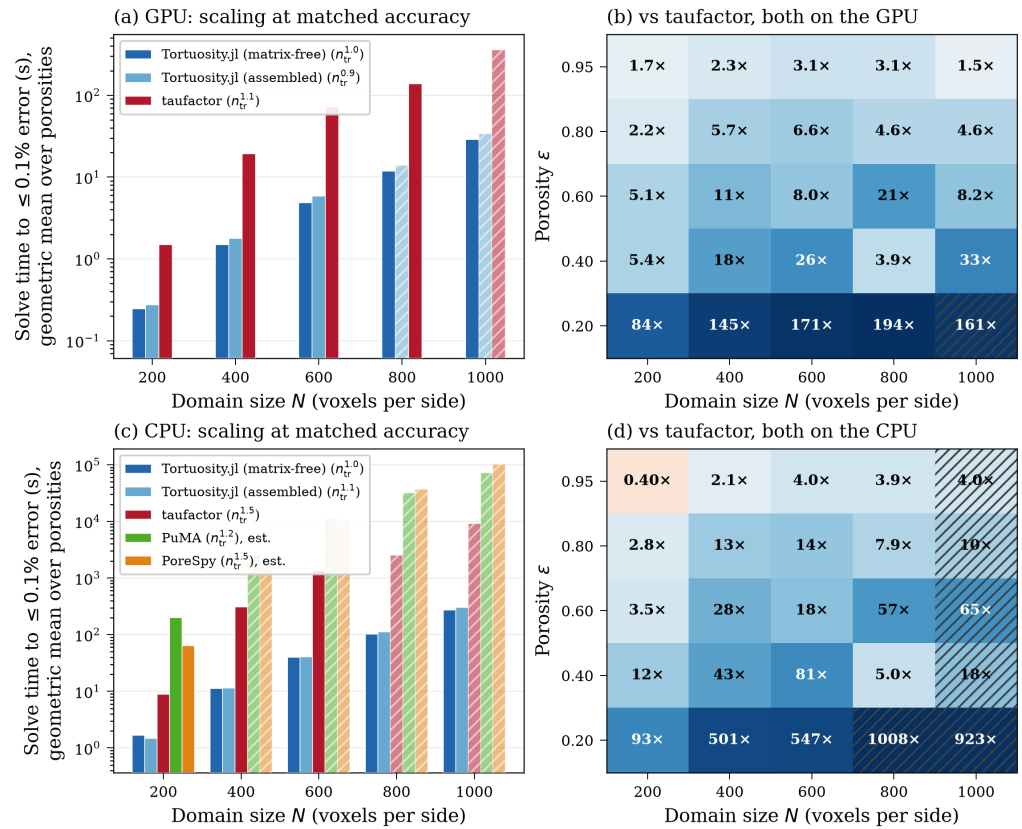


Figure 2: Time to reach at most 0.1% relative error in τ for the reference feature size. The scaling panels retain the same five porosities at every size; missing target times are projected from the same case family's measured size trend so that nonconverged cases do not bias the geometric mean downward. Solid bars show measured size sweeps, while hatched bars denote projections. For the assembled GPU method, hatched bars at the two largest sizes are hypothetical timing projections for cases that exceeded device memory; they preserve the geometric-mean assembled-to-matrix-free timing ratio at the shared measured sizes. Legend exponents describe empirical scaling with the recorded number of transporting voxels, n_{tr} , with "est." marking exponents derived from size probes rather than fitted sweeps. Panels (b) and (d) show speed ratios against taufactor on the GPU and CPU, respectively. Each cell is taufactor's time divided by the Tortuosity.jl time, so values above one favor Tortuosity.jl and values below one favor taufactor. Hatched cells mark projections: where taufactor was not run, or did not reach the target before its timeout, its time comes from the same per-porosity power law the scaling panels use, so the two agree cell for cell, and is held at or above any time it had already spent, which is a measured lower bound.

110 Limitations

111 The main input-related limitation is that disconnected pore clusters are retained as stagnant
 112 volume; users who want transport only through the percolating phase must trim these clusters
 113 before solving. The performance evidence is also deliberately bounded: the benchmarks
 114 use one machine, one family of synthetic microstructures, and reference values computed
 115 by the package's double-precision CPU solver. Among the GPU backends, only CUDA was
 116 benchmarked. The timing sweeps for PuMA and PoreSpy were limited to 200³; their larger
 117 dashed values in Figure 2 are projections calibrated by separate size probes. The results
 118 should therefore be interpreted as representative comparisons rather than universal performance
 119 rankings.

Research impact statement

The benchmark results show that `Tortuosity.jl` can analyze billion-voxel images on a single workstation GPU while reducing run time relative to the benchmarked alternatives in regimes where large-image calculations become costly. The package is registered in Julia's General registry, has been developed publicly by multiple contributors since 2023, and provides tutorials, API documentation, automated tests, and contribution guidelines. The complete benchmark workflow is distributed with the package, including pinned software environments, image checksums, raw result tables, and scripts that regenerate the reported analyses and figures. Together, these resources provide a practical tool for large image-based transport studies and inspectable evidence supporting its correctness and performance.

AI usage disclosure

Generative AI tools (Claude Opus 5 and GPT-5.6 Sol) were used for code suggestions and review, test and benchmark scaffolding, documentation, and manuscript drafting and editing. The human authors reviewed, edited, tested, and validated all AI-assisted outputs and made the core algorithmic, scientific, and editorial decisions.

Acknowledgements

We thank the developers of `TauFactor`, whose work informed this package, and the Julia community for maintaining the ecosystem on which `Tortuosity.jl` depends. This work received no external funding.

References

- Besard, T., Foket, C., & De Sutter, B. (2019). Effective extensible programming: Unleashing Julia on GPUs. *IEEE Transactions on Parallel and Distributed Systems*, 30(4), 827–841. <https://doi.org/10.1109/TPDS.2018.2872064>
- Churavy, V., & contributors. (2024). *KernelAbstractions.jl: Heterogeneous programming in Julia*. <https://doi.org/10.5281/zenodo.4021259>
- Cooper, S. J., Bertei, A., Shearing, P. R., Kilner, J. A., & Brandon, N. P. (2016). `TauFactor`: An open-source application for calculating tortuosity factors from tomographic data. *SoftwareX*, 5, 203–210. <https://doi.org/10.1016/j.softx.2016.09.002>
- Ferguson, J. C., Panerai, F., Borner, A., & Mansour, N. N. (2018). `PuMA`: The Porous Microstructure Analysis software. *SoftwareX*, 7, 81–87. <https://doi.org/10.1016/j.softx.2018.03.001>
- Ferguson, J. C., Semeraro, F., Thornton, J. M., Panerai, F., Borner, A., & Mansour, N. N. (2021). Update 3.0 to `PuMA`: The Porous Microstructure Analysis software. *SoftwareX*, 15, 100775. <https://doi.org/10.1016/j.softx.2021.100775>
- Fu, J., Thomas, H. R., & Li, C. (2021). Tortuosity of porous media: Image analysis and physical simulation. *Earth-Science Reviews*, 212, 103439. <https://doi.org/10.1016/j.earscirev.2020.103439>
- Ghanbarian, B., Hunt, A. G., Ewing, R. P., & Sahimi, M. (2013). Tortuosity in porous media: A critical review. *Soil Science Society of America Journal*, 77(5), 1461–1477. <https://doi.org/10.2136/sssaj2012.0435>
- Gostick, J. T., Khan, Z. A., Tranter, T. G., Kok, M. D. R., Agnaou, M., Sadeghi, M. A., & Jervis, R. (2019). `PoreSpy`: A Python toolkit for quantitative analysis of porous media images.

- 162 *Journal of Open Source Software*, 4(37), 1296. <https://doi.org/10.21105/joss.01296>
- 163 Kench, S., Squires, I., & Cooper, S. J. (2023). TauFactor 2: A GPU accelerated python
164 tool for microstructural analysis. *Journal of Open Source Software*, 8(88), 5358. <https://doi.org/10.21105/joss.05358>
- 165
- 166 Kimmerer, W., Puri, V., Ambaliya, J., & Rackauckas, C. (2024). *LinearSolve.jl: Unified*
167 *interface for linear solvers in Julia*. <https://github.com/SciML/LinearSolve.jl>
- 168 Rackauckas, C., & Nie, Q. (2017). DifferentialEquations.jl – a performant and feature-rich
169 ecosystem for solving differential equations in Julia. *Journal of Open Research Software*,
170 5(1), 15. <https://doi.org/10.5334/jors.151>
- 171 Tjaden, B., Cooper, S. J., Brett, D. J. L., Kramer, D., & Shearing, P. R. (2016). On the
172 origin and application of the Bruggeman correlation for analysing transport phenomena
173 in electrochemical systems. *Current Opinion in Chemical Engineering*, 12, 44–51. <https://doi.org/10.1016/j.coche.2016.02.006>
- 174

DRAFT